

Reviewer Pack — Minimal Order of K-theory Classes and Resolution Proof Width...

Entry 402d569decef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 04:22:00 UTC

Minimal Order of K-theory Classes and Resolution Proof Width Correlation

Entry ID: 402d569decef **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 04:22:00 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean formula φ with n variables, the minimal order of a class in the algebraic K-theory of the polynomial ring corresponding to φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\min_order(K(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

The algebraic K-theory provides an invariant for studying the structure of rings, which may capture non-trivial aspects of the complexity inherent in Boolean formulas. If true, this conjecture would establish a link between high-level algebraic structures and the practical complexities of proof systems.

Taxonomy category: algebraic_k_theory_resolution_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 593657ecf9e5612a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for a given Boolean formula φ with n variables, Pearson's correlation coefficient between the minimal order of a class in the algebraic K-theory ($\min_order(K(\varphi))$) and its resolution proof width ($w(\varphi)$) over 30 random seeds is ≥ 0.8 AND the mean metric value across all seeds does not exceed 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Algebraic K-theory' AND 'resolution proof complexity' AND 'minimal order' - 'K-theory classes' AND 'polynomial ring' AND 'resolution proof width' - 'algebraic K-theory' AND 'correlation' AND 'proof complexity'

Top relevant hits considered: - [http://arxiv.org/abs/1311.1421v3] Multiplicative differential algebraic K-theory and applications - [http://arxiv.org/abs/1510.06118v3] G-theory of root stacks and equivariant K-theory - [http://arxiv.org/abs/1208.1599v1] Algebraic K-theory of endomorphism rings - [http://arxiv.org/abs/1811.09564v3] Dévissage for Waldhausen K-theory - [http://arxiv.org/abs/1710.00331v2] Hecke modules for arithmetic groups via bivariant K-theory - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e} -linear maps - [http://arxiv.org/abs/1812.11710v1] Geometric Satake correspondence for affine Kac-Moody Lie algebras of type A - [http://arxiv.org/abs/1705.03356v2] Growth in varieties of multioperator algebras and Groebner bases in operads

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda k: abs(A[k][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_multiply(A, B):
```

```

m, n, p = len(A), len(B), len(B[0])
C = [[0] * p for _ in range(m)]
for i in range(m):
    for j in range(p):
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if m == 1:
        return A[0][0]
    det = Fraction(0)
    for j in range(n):
        submatrix = [[A[i][k] for k in range(n) if k != j] for i in range(1, m)]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def polynomial_ring_from_cnf(cnf):
    variables = set()
    for clause in cnf:
        for literal in clause:
            if literal < 0:
                literal = -literal
            variables.add(literal)
    n = len(variables)
    ring_size = 2**n
    A = [[Fraction(0)] * ring_size for _ in range(ring_size)]
    for i in range(ring_size):
        for j in range(ring_size):
            if (i & j) == 0:
                A[i][j] = Fraction(1)
    return A

def min_order_k_theory(A):
    rank = 0
    for row in gaussian_elimination(A):
        if any(row):
            rank += 1
    return rank

def resolution_proof_width(cnf):
    clauses = len(cnf)
    max_clause_length = max(len(clause) for clause in cnf)
    return clauses + max_clause_length - 2

n_values = [5, 10, 15, 20, 30, 40]
results = []
total_metric_value = Fraction(0)
instances_tested = 0
n_max = 0

for n in n_values:
    for _ in range(5):

```

```

        cnf = [[random.randint(1, n) if random.choice([True, False]) else -random.randint(1, n) for _ in range(n)]
        A = polynomial_ring_from_cnf(cnf)
        min_order = min_order_k_theory(A)
        w_phi = resolution_proof_width(cnf)
        results.append((min_order, w_phi))
        instances_tested += 1
        n_max = max(n_max, n)

    if not results:
        return {
            "metric_name": "Pearson's correlation coefficient",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    min_orders, w_phis = zip(*results)
    mean_min_order = sum(min_orders) / len(min_orders)
    mean_w_phi = sum(w_phis) / len(w_phis)

    correlation_coefficient = sum((x - mean_min_order) * (y - mean_w_phi) for x, y in results) / (len(results) - 1)

    return {
        "metric_name": "Pearson's correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient >= 0.8 and abs(correlation_coefficient) <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b82b14fd.py", line 133, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b82b14fd.py", line 96, in run_trial
    min_order = min_order_k_theory(A)
               ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b82b14fd.py", line 76, in min_order_k_t
    for row in gaussian_elimination(A):
               ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b82b14fd.py", line 25, in gaussian_elim
    A[i], A[max_row] = A[max_row], A[i]
                       ~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required calculations to determine if the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to allow for the completion of the necessary calculations.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21920
2	propose	ollama_remote	glm4:latest	0	0	19720
3	preregistration	ollama_remote	glm4:latest	0	0	11083
4	novelty	ollama_remote	glm4:latest	0	0	11716
5	novelty	ollama_remote	glm4:latest	0	0	9590
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18441
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19888
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11169
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15614
10	judge	ollama_remote	glm4:latest	0	0	11562

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 150705 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/402d569decef.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/402d569decef.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/402d569decef.tar.gz (if generated)